

4.3 - The Latin Square Design

4.3 - The Latin Square Design

Latin Square Designs are probably not used as much as they should be - they are very efficient designs. Latin square designs allow for two blocking factors. In other words, these designs are used to simultaneously control (or eliminate) **two sources of nuisance variability**. For instance, if you had a plot of land the fertility of this land might change in both directions, North -- South and East -- West due to soil or moisture gradients. So, both rows and columns can be used as blocking factors. However, you can use Latin squares in lots of other settings. As we shall see, Latin squares can be used as much as the RCBD in industrial experimentation as well as other experiments.

Whenever, you have more than one blocking factor a Latin square design will allow you to remove the variation for these two sources from the error variation. So, consider we had a plot of land, we might have blocked it in columns and rows, i.e. each row is a level of the row factor, and each column is a level of the column factor. We can remove the variation from our measured response in both directions if we consider both rows and columns as factors in our design.

The Latin Square Design gets its name from the fact that we can write it as a square with Latin letters to correspond to the treatments. The treatment factor levels are the Latin letters in the Latin square design. The number of rows and columns has to correspond to the number of treatment levels. So, if we have four treatments then we would need to have four rows and four columns in order to create a Latin square. This gives us a design where we have each of the treatments and in each row and in each column.

	columns				
	A	B	C	D	Each treatment occurs in every column and row
rows	B	C	D	A	
	C	D	A	B	
	D	A	B	C	

This is just one of many 4×4 squares that you could create. In fact, you can make any size square you want, for any number of treatments - it just needs to have the following property associated with it - that each treatment occurs only once in each row and once in each column.

Consider another example in an industrial setting: the rows are the batch of raw material, the columns are the operator of the equipment, and the treatments (A, B, C and D) are an industrial process or protocol for producing a particular product.

What is the model? We let:

$$y_{ijk} = \mu + \rho_i + \beta_j + \tau_k + e_{ijk}$$

$$i = 1, \dots, t$$

$$j = 1, \dots, t$$

$[k = 1, \dots, t]$ where $k = d(i, j)$ and the total number of observations

$N = t^2$ (the number of rows times the number of columns) and t is the number of treatments.

Note that a Latin Square is an incomplete design, which means that it does not include observations for all possible combinations of i, j and k . This is why we use notation

$k = d(i, j)$. Once we know the row and column of the design, then the treatment is specified. In other words, if we know i and j , then k is specified by the Latin Square design.

This property has an impact on how we calculate means and sums of squares, and for this reason, we can not use the balanced ANOVA command in Minitab even though it looks perfectly balanced. We will see later that although it has the property of orthogonality, you still cannot use the balanced ANOVA command in Minitab because it is not complete.

An assumption that we make when using a Latin square design is that the three factors (treatments, and two nuisance factors) **do not interact**. If this assumption is violated, the Latin Square design error term will be inflated.

The randomization procedure for assigning treatments that you would like to use when you actually apply a Latin Square, is somewhat restricted to preserve the structure of the Latin Square. The ideal randomization would be to select a square from the set of all possible Latin squares of the specified size. However, a more practical randomization scheme would be to select a standardized Latin square at random (these are tabulated) and then:

1. randomly permute the columns,
2. randomly permute the rows, and then
3. assign the treatments to the Latin letters in a random fashion.

Consider a factory setting where you are producing a product with 4 operators and 4 machines. We call the columns the operators and the rows the machines. Then you can randomly assign the specific operators to a row and the specific machines to a column. The treatment is one of four protocols for producing the product and our interest is in the average time needed to produce each product. If both the machine and the operator have an effect on the time to produce, then by using a Latin Square Design this variation due to machine or operators will be effectively removed from the analysis.

The following table gives the degrees of freedom for the terms in the model.

AOV	df	df for the example
Rows	$t-1$	3

Cols	$t-1$	3
Treatments	$t-1$	3
Error	$(t-1)(t-2)$	6
Total	$(t^2 - 1)$	15

A Latin Square design is actually easy to analyze. Because of the restricted layout, one observation per treatment in each row and column, the model is orthogonal.

If the row, ρ_i , and column, β_j , effects are random with expectations zero, the expected value of Y_{ijk} is $\mu + \tau_k$. In other words, the treatment effects and treatment means are orthogonal to the row and column effects. We can also write the sums of squares, as seen in Table 4.10 in the text.

We can test for row and column effects, but our focus of interest in a Latin square design is on the treatments. Just as in RCBD, the row and column factors are included to reduce the error variation but are not typically of interest. And, depending on how we've conducted the experiment they often haven't been randomized in a way that allows us to make any reliable inference from those tests.

Note: if you have missing data then you need to use the general linear model and test the effect of treatment after fitting the model that would account for the row and column effects.

In general, the General Linear Model tests the hypothesis that:

$$H_0 : \tau_i = 0 \text{ vs. } H_A : \tau_i \neq 0$$

To test this hypothesis we will look at the F -ratio which is written as:

$$F = \frac{MS(\tau_k | \mu, \rho_i, \beta_j)}{MSE(\mu, \rho_i, \beta_j, \tau_k)} \sim F((t-1), (t-1)(t-2))$$

To get this in Minitab you would use GLM and fit the three terms: rows, columns and treatments. The F statistic is based on the adjusted MS for treatment.

The Rocket Propellant Problem – A Latin Square Design

Batches of Raw Material	Operators				
	1	2	3	4	5
1	A = 24	B = 20	C = 19	D = 24	E = 24
2	B = 17	C = 24	D = 30	E = 27	A = 36
3	C = 18	D = 38	E = 26	A = 27	B = 21
4	D = 26	E = 31	A = 26	B = 23	C = 22
5	E = 22	A = 30	B = 20	C = 29	D = 31

Latin Square Design for the Rocket Propellant

Statistical Analysis of the Latin Square Design

The statistical (effects) model is:

$$Y_{ijk} = \mu + \rho_i + \beta_j + \tau_k + \varepsilon_{ijk} \quad \left\{ \begin{array}{l} i = 1, 2, \dots, p \\ j = 1, 2, \dots, p \\ k = 1, 2, \dots, p \end{array} \right.$$

but $k = d(i, j)$ shows the dependence of k in the cell i, j on the design layout, and $p = t$ the number of treatment levels.

The statistical analysis (ANOVA) is much like the analysis for the RCBD.

The analysis for the rocket propellant example is presented in Example 4.3.

4.4 - Replicated Latin Squares

4.4 - Replicated Latin Squares

Latin Squares are very efficient by including two blocking factors, however, the *d.f.* for error are often too small. In these situations, we consider replicating a Latin Square. Let's go back to the factory scenario again as an example and look at $n = 3$ repetitions of a 4×4 Latin square.

We labeled the row factor the machines, the column factor the operators and the Latin letters denoted the *protocol* used by the operators which were the treatment factor. We will replicate this Latin Square experiment $n = 3$ times. Now we have total observations equal to

$$N = t^2 n.$$

You could use the same squares over again in each replicate, but we prefer to randomize these separately for each replicate. It might look like this:

		operators			
		1	2	3	4
machines	1	A	B	C	D
	2	B	C	D	A
	3	C	D	A	B
	4	D	A	B	C
Rep 1					
		operators			
		1	2	3	4
machines	1	D	A	B	C
	2	A	B	C	D
	3	B	C	D	A
	4	C	D	A	B
Rep 2					
		operators			
		1	2	3	4
machines	1	C	D	A	B
	2	D	A	B	C
	3	A	B	C	D
	4	B	C	D	A
Rep 3					

Ok, with this scenario in mind, let's consider three cases that are relevant and each case requires a different model to analyze. The cases are determined by whether or not the blocking factors are the same or different across the replicated squares. The treatments are going to be the same but the question is whether the levels of the blocking factors remain the same.

Case 1

Here we will have the same row and column levels. For instance, we might do this experiment all in the same factory using the same machines and the same operators for these machines. The first replicate would occur during the first week, the second replicate would occur during the second week, etc. Week one would be replication one, week two would be replication two and week three would be replication three.

We would write the model for this case as:

$$Y_{hijk} = \mu + \delta_h + \rho_i + \beta_j + \tau_k + e_{hijk}$$

where:

$$\begin{aligned} h &= 1, \dots, n \\ i &= 1, \dots, t \\ j &= 1, \dots, t \\ k &= d_h(i, j) - \text{the Latin letters} \end{aligned}$$

This is a simple extension of the basic model that we had looked at earlier. We have added one more term to our model. The row and column and treatment all have the same parameters, the same effects that we had in the single Latin square. In a Latin square, the error is a combination of any interactions that might exist and experimental error. Remember, we can't estimate interactions in a Latin square.

Let's take a look at the analysis of variance table.

AOV	df	df for Case 1	SS
rep=week	$n - 1$	2	

row=machine	$t - 1$	3
column=operator	$t - 1$	3
treatment=protocol	$t - 1$	3
error	$(t - 1) [n(t + 1) - 3]$	36
Total	$nt^2 - 1$	47

Case 2

In this case, one of our blocking factors, either row or column, is going to be the same across replicates whereas the other will take on new values in each replicate. Back to the factory example e.g., we would have a situation where the machines are going to be different (you can say they are nested in each of the repetitions) but the operators will stay the same (crossed with replicates). In this scenario, perhaps, this factory has three locations and we want to include machines from each of these three different factories. To keep the experiment standardized, we will move our operators with us as we go from one factory location to the next. This might be laid out like this:

		operators						operators						operators			
		1	2	3	4			1	2	3	4			1	2	3	4
machines	1	A	B	C	D	machines	5	D	A	B	C	machines	9	C	D	A	B
	2	B	C	D	A		6	A	B	C	D		10	D	A	B	C
	3	C	D	A	B		7	B	C	D	A		11	A	B	C	D
	4	D	A	B	C		8	C	D	A	B		12	B	C	D	A
Rep 1 Factory 1						Rep 2 Factory 2						Rep 3 Factory 3					

There is a subtle difference here between this experiment in a Case 2 and the experiment in Case 1 - but it does affect how we analyze the data. Here the model is written as:

$$Y_{hijk} = \mu + \delta_h + \rho_{i(h)} + \beta_j + \tau_k + e_{hijk}$$

where:

$$\begin{aligned}
 h &= 1, \dots, n \\
 i &= 1, \dots, t \\
 j &= 1, \dots, t \\
 k &= d_h(i, j) - \text{the Latin letters}
 \end{aligned}$$

and the 12 machines are distinguished by nesting the i index within the h replicates.

This affects our ANOVA table. Compare this to the previous case:

AOV	df	df for Case 2	SS
rep = factory	$n - 1$	2	See text p. 144.
row (rep) = machine (factory)	$n (t - 1)$	9	
column = operator	$t - 1$	3	
treatment = protocol	$t - 1$	3	
error	$(t - 1) (nt - 2)$	30	
Total	$nt^2 - 1$	47	

Note that Case 2 may also be flipped where you might have the same machines, but different operators.

Case 3

In this case, we have different levels of both the row and the column factors. Again, in our factory scenario, we would have different machines **and** different operators in the three replicates. In other words, both of these factors would be nested within the replicates of the experiment.

operators					operators					operators							
1 2 3 4					5 6 7 8					9 10 11 12							
machines	1	A	B	C	D	machines	5	D	A	B	C	machines	9	C	D	A	B
	2	B	C	D	A		6	A	B	C	D		10	D	A	B	C
	3	C	D	A	B		7	B	C	D	A		11	A	B	C	D
	4	D	A	B	C		8	C	D	A	B		12	B	C	D	A
Rep 1 Factory 1					Rep 2 Factory 2					Rep 3 Factory 3							

We would write this model as:

$$Y_{hijk} = \mu + \delta_h + \varrho_{i(h)} + \beta_{j(h)} + \tau_k + e_{hijk}$$

where:

$$h = 1, \dots, n$$

$$i = 1, \dots, t$$

$$j = 1, \dots, t$$

$k = d_h(i, j)$ - the Latin letters

Here we have used nested terms for both of the block factors representing the fact that the levels of these factors are not the same in each of the replicates.

The analysis of variance table would include:

AOV	df	df for Case 3	SS
rep = factory	$n - 1$	2	See text p. 144.
row (rep) = machine (factory)	$n (t - 1)$	9	
column (rep) = operator (factory)	$n (t - 1)$	9	
treatment protocol	$t - 1$	3	
error	$(t - 1) [n (t - 1) - 1]$	24	
Total	$nt^2 - 1$	47	

Which case is best?

There really isn't a best here... the choice of case depends on how you need to conduct the experiment. If you are simply replicating the experiment with the same row and column levels, you are in Case 1. If you are changing one or the other of the row or column factors, using different machines or operators, then you are in Case 2. If both of the block factors have levels that differ across the replicates, then you are in Case 3. The third case, where the replicates are different factories, can also provide a comparison of the factories. The fact that you are replicating Latin Squares does allow you to estimate some interactions that you can't estimate from a single Latin Square. If we added a treatment by factory interaction term, for instance, this would be a meaningful term in the model, and would inform the researcher whether the same protocol is best (or not) for all the factories.

The degrees of freedom for error grows very rapidly when you replicate Latin squares. But usually if you are using a Latin Square then you are probably not worried too much about this error. The error is more dependent on the specific conditions that exist for performing the experiment. For instance, if the protocol is complicated and training the operators so they can

conduct all four becomes an issue of resources then this might be a reason why you would bring these operators to three different factories. It depends on the conditions under which the experiment is going to be conducted.

Situations where you should use a Latin Square are where you have a **single** treatment factor and you have two blocking or nuisance factors to consider, which can have the same number of levels as the treatment factor.

4.5 - What do you do if you have more than 2 blocking factors?

4.5 - What do you do if you have more than 2 blocking factors?

When might this occur? Let's consider the factory example again. In this factory you have four machines and four operators to conduct your experiment. You want to complete the experimental trials in a week. Use the animation below to see how this example of a typical treatment schedule pans out.

<https://www.youtube.com/watch/qGBk0EG2BVo> ^[3]

As the treatments were assigned you should have noticed that the treatments have become confounded with the days. Days of the week are not all the same, Monday is not always the best day of the week! Just like any other factor not included in the design you hope it is not important or you would have included it into the experiment in the first place.

What we now realize is that two blocking factors is not enough! We should also include the day of the week in our experiment. It looks like day of the week could affect the treatments and introduce bias into the treatment effects, since not all treatments occur on Monday. We want a design with 3 blocking factors; machine, operator, and day of the week.

One way to do this would be to conduct the entire experiment on one day and replicate it four times. But this would require $4 \times 16 = 64$ observations not just 16. Or, we could use what is called a Graeco-Latin Square.

Graeco-Latin Squares

We write the Latin square first then each of the Greek letters occurs alongside each of the Latin letters. A Graeco-Latin square is a set of two orthogonal Latin squares where each of the Greek and Latin letters is a Latin square and the Latin square is orthogonal to the Greek square. Use the animation below to explore a Graeco-Latin square:

<https://www.youtube.com/watch/tZTUMSV-OdY> ^[4]

The Greek letters each occur one time with each of the Latin letters. A Graeco-Latin square is orthogonal between rows, columns, Latin letters and Greek letters. It is completely orthogonal.

How do we use this design?

We let the row be the machines, the column be the operator, (just as before) and the Greek letter the day, (you could also think of this as the order in which it was produced). Therefore the Greek letter could serve the multiple purposes as the day effect or the order effect. The Latin

letters are assigned to the treatments as before.

We want to account for all three of the blocking factor sources of variation, and remove each of these sources of error from the experiment. Therefore we must include them in the model.

Here is the model for this design:

$$Y_{ijkl} = \mu + \alpha_i + \beta_j + \tau_k + \gamma_l + e_{ijkl}$$

So, we have three blocking factors and one treatment factor.

and i, j, k and l all go from $1, \dots, t$, where i and j are the row and column indices, respectively, and k and l are defined by the design, that is, k and l are specified by the Latin and Greek letters, respectively.

This is a highly efficient design with $N = t^2$ observations.

You could go even farther and have more than two orthogonal latin squares together. These are referred to a Hyper-Graeco-Latin squares!

[Fisher, R.A. The Design of Experiments, 8th edition, 1966, p.82-84](#), gives examples of hyper-Graeco-Latin squares for $t = 4, 5, 8$ and 9 . ^[5]

NOTE! It is impossible to have a 6×6 Graeco-Latin square! So in designing your experiment with a Graeco-Latin Square - don't have 6 treatments! Add another, or drop one!

4.7 - Incomplete Block Designs

4.7 - Incomplete Block Designs

In using incomplete block designs we will use the notation $t = \#$ of treatments. We define the block size as k . And, as you will see, in incomplete block designs k will be less than t . You cannot assign all of the treatments in each block. In short,

$t = \#$ of treatments,

$k =$ block size,

$b = \#$ of blocks,

$r_i = \#$ of replicates for treatment i , in the entire design.

Remember that an equal number of replications is the best way to be sure that you have minimum variance if you're looking at all possible pairwise comparisons. If $r_i = r$ for all treatments, the total number of observations in the experiment is N where:

$$N = t (r) = b (k)$$

The incidence matrix which defines the design of the experiment, gives the number of observations say n_{ij} for the i^{th} treatment in the j^{th} block. This is what it might look like here:

		blocks			
	n_{ij}	1	2	...	b
treatments	1				
	2				
	.				
	.				
	t				

Here we have treatments 1, 2, up to t and the blocks 1, 2, up to b . For a complete block design, we would have each treatment occurring one time within each block, so all entries in this matrix would be 1's. For an incomplete block design, the incidence matrix would be 0's and 1's simply indicating whether or not that treatment occurs in that block.

Example 1

The example that we will look at is Table 4.22 (4.21 in 7th ed). Here is the incidence matrix for this example:

	1	2	3	4
1	1	1	0	1
2	0	1	1	1
3	1	1	1	0
4	1	0	1	1

Here we have $t = 4$, $b = 4$, (four rows and four columns) and $k = 3$ (so at each block we can only put three of the four treatments leaving one treatment out of each block). So, in this case, the row sums (r_i) and the columns sums, k , are all equal to 3.

	1	2	3	4	
1	1	1	0	1	3
2	0	1	1	1	3
3	1	1	1	0	3
4	1	0	1	1	3
	3	3	3	3	

In general, we are faced with a situation where the number of treatments is specified, and the block size, or number of experimental units per block (k) is given. This is usually a constraint given from the experimental situation. And then, the researcher must decide how many blocks are needed to run and how many replicates that provides in order to achieve the precision or the power that you want for the test.

Example 2

Here is another example of an incidence matrix for allocating treatments and replicates in an incomplete block design. Let's take an example where $k = 2$, still $t = 4$, and $b = 4$. That gives us a case $r = 2$. In This case we could design our incidence matrix so that it might look like this:

	1	2	3	4	
1	1	0	1	0	2
2	0	1	0	1	2
3	1	0	1	0	2
4	0	1	0	1	2
	2	2	2	2	

This example has two observations per block so $k = 2$ in each case and for all treatments $r = 2$.

Balanced Incomplete Block Design (BIBD)

A BIBD is an incomplete block design where all pairs of treatments occur together within a block an equal number of times (λ). In general, we will specify $\lambda_{ii'}$ as the number of times treatment i occurs with i' , in a block.

Let's look at previous cases. How many times does treatment one and two occur together in this first example design?

	blocks				
	1	2	3	4	
1	1	1	0	1	3
2	0	1	1	1	3
3	1	1	1	0	3
4	1	0	1	1	3
	3	3	3	3	

It occurs together in block 2 and then again in block 4 (highlighted in light blue). So, $\lambda_{12} = 2$. If we look at treatment one and three, this occurs together in block one and in block two therefore $\lambda_{13} = 2$. In this design, you can look at all possible pairs. Let's look at 1 and 4 - they occur together twice, 2 and 3 occur together twice, 2 and 4 twice, and 3 and 4 occur together twice. For this design $\lambda_{ii'} = 2$ for all ii' treatment pairs defining the concept of balance in this incomplete block design.

If the number of times treatments occur together within a block is equal across the design for all pairs of treatments then we call this a balanced incomplete block design (BIBD).

Now look at the incidence matrix for the second example.

	1	2	3	4	
1	1	0	1	0	2
2	0	1	0	1	2
3	1	0	1	0	2
4	0	1	0	1	2
	2	2	2	2	

We can see that:

λ_{12} occurs together 0 times.

λ_{13} occurs together 2 times.

λ_{14} occurs together 0 times.

λ_{23} occurs together 0 times.

λ_{24} occurs together 2 times.

λ_{34} occurs together to 0 times.

Here we have two pairs occurring together 2 times and the other four pairs occurring together 0 times. Therefore, this is not a balanced incomplete block design (BIBD).

What else is there about BIBD?

We can define λ in terms of our design parameters when we have equal block size k , and equal replication $r_i = r$. For a given set of t , k , and r we define λ as:

$$\lambda = r (k - 1) / t - 1$$

So, for the first example that we looked at earlier - let's plug in the values and calculate λ :

$$\lambda = 3 (3 - 1) / (4 - 1) = 2$$

Here is the key: when λ is equal to an integer number it tells us that a balanced incomplete block design exists. Let's look at the second example and use the formula and plug in the values for this second example. So, for $t = 4$, $k = 2$, $r = 2$ and $b = 4$, we have:

$$\lambda = 2 (2 - 1) / (4 - 1) = 0.666$$

Since λ is not an integer there does not exist a balanced incomplete block design for this experiment. We would either need more replicates or a larger block size. Seeing as how the block size in this case is fixed, we can achieve a balanced complete block design by adding more replicates so that λ equals at least 1. It needs to be a whole number in order for the design to be balanced.

We will talk about partially balanced designs later. But in thinking about this case we note that a balanced design doesn't exist so what would be the best partially balanced design? That would be a question that you would ask if you could only afford four blocks and the block size is two. Given this situation, is the design in Example 2 the best design we can construct? The best partially balanced design is where λ_{ii} should be the nearest integers to the λ that we calculated. In our case each λ_{ii} should be either 0 or 1, the integers nearest 0.667. This example is not as close to balanced as it could be. In fact, it is not even a connected design where you can go from any treatment to any other treatment within a block. More about this later...

How do you construct a BIBD?

In some situations, it is easy to construct the best IBD, however, for other cases it can be quite difficult and we will look them up in a reference.

Let's say that we want six blocks, we still want 4 treatments and our block size is still 2.

Calculate $\lambda = r (k - 1) / (t - 1) = 1$. We want to create all possible pairs of treatments because lambda is equal to one. We do this by looking at all possible combinations of four treatments taking two at a time. We could set up the incidence matrix for the design or we could represent it like this - entries in the table are treatment labels: {1, 2, 3, 4}.

	1	2	3	4	5	6
1	1	1	1	2	2	3
2	2	3	4	3	4	4

However, this method of constructing a BIBD using all possible combinations, does not always work as we now demonstrate. If the number of combinations is too large then you need to find a subset - - not always easy to do. However, sometimes you can use Latin Squares to

construct a BIBD. As an example, let's take any 3 columns from a 4×4 Latin Square design. This subset of columns from the whole Latin Square creates a BIBD. However, not every subset of a Latin Square is a BIBD.

Let's look at an example. In this example we have $t = 7$, $b = 7$, and $k = 3$. This means that $r = 3 = (bk) / t$. Here is the 7×7 Latin square :

A	B	C	D	E	F	G
B	C	D	E	F	G	A
C	D	E	F	G	A	B
D	E	F	G	A	B	C
E	F	G	A	B	C	D
F	G	A	B	C	D	E
G	A	B	C	D	E	F

We want to select ($k = 3$) three columns out of this design where each treatment occurs **once** with every other treatment because

$$\lambda = 3(3 - 1) / (7 - 1) = 1.$$

We could select the first three columns - let's see if this will work. Click the animation below to see whether using the first three columns would give us combinations of treatments where treatment pairs are not repeated.

Since the first three columns contain some pairs more than once, let's try columns 1, 2, and now we need a third...how about the fourth column. If you look at all possible combinations in each row, each treatment pair occurs only one time.

What if we could afford a block size of 4 instead of 3? Here $t = 7$, $b = 7$, $k = 4$, then $r = 4$. We calculate $\lambda = r(k - 1) / (t - 1) = 2$ so a BIBD does exist. For this design with a block size of 4 we can select 4 columns (or rows) from a Latin square. Let's look at columns again... can you select the correct 4?

Now consider the case with 8 treatments. The number of possible combinations of 8 treatments taking 4 at a time is 70. Thus with 70 sets of 4 from which you have to choose 14 blocks - - wow, this is a big job! At this point, we should simply look at an appropriate reference. Here is a handout - [a catalog that will help you with this selection process](#) - taken from Cochran & Cox, *Experimental Design*, p. 469-482. ^[13]

Analysis of BIBD's

When we have missing data, it affects the average of the remaining treatments in a row, i.e., when complete data does not exist for each row - this affects the means. When we have complete data the block effect and the column effects both drop out of the analysis since they

are orthogonal. With missing data or IBDs that are not orthogonal, even BIBD where orthogonality does not exist, the analysis requires us to use GLM which codes the data like we did previously. The GLM fits first the block and then the treatment.

The sequential sums of squares (Seq SS) for block is not the same as the Adj SS.

We have the following:

Seq SS

$$SS(\beta | \mu) = 55.0$$

$$SS(\tau | \mu, \beta) = 22.50$$

Adj SS

$$SS(\beta | \mu, \tau) = 66.08$$

$$SS(\tau | \mu, \beta) = 22.75$$

Switch them around...now first fit treatments and then the blocks.

Seq SS

$$SS(\tau | \mu) = 11.67$$

$$SS(\beta | \mu, \tau) = 66.08$$

Adj SS

$$SS(\tau | \mu, \beta) = 22.75$$

$$SS(\beta | \mu, \tau_i) = 66.08$$

The 'least squares means' come from the fitted model. Regardless of the pattern of missing data or the design we can conceptually think of our design represented by the model:

$$Y_{ij} = \mu + \beta_i + \tau_j + e_{ij}$$

$$i = 1, \dots, b, \quad j = 1, \dots, t$$

You can obtain the 'least squares means' from the estimated parameters from the least squares fit of the model.

Optional Section

See the discussion in the text for *Recovery of Interblock Information*, p. 154. This refers to a procedure which allows us to extract additional information from a BIBD when the blocks are a random effect. Optionally you can read this section. We illustrate the analysis by the use of the software, PROC Mixed in SAS ([L03_sas_Ex_4_5.sas](#)): ^[14]


```

data; input blk trt Y; cards;
1 1 73
1 3 73
1 4 75
2 1 74
2 2 75
2 3 75
3 2 67
3 3 68
3 4 72
4 1 71
4 2 72
4 4 75
;;;;
/*This data is from Example 4-5 in Montgomery, Design and Analysis of experiments, 6th
edition, */
/* Wiley, 2005, pages 147-154. This demonstrates the recovery of interblock
information when */
/* the blocks are considered random. */
proc glm; class trt blk;
model Y = blk trt;
lsmeans trt/ e stderr pdiff;

proc mixed; class trt blk;
model Y = trt;
random blk;
lsmeans trt/ e pdiff;
/* The next 4 estimate statements calculate the treatment effects from the solution*/
estimate "trt effect 1" trt +.75 -.25 -.25 -.25/e;
estimate "trt effect 2" trt -.25 +.75 -.25 -.25/e;
estimate "trt effect 3" trt -.25 -.25 +.75 -.25/e;
estimate "trt effect 4" trt -.25 -.25 -.25 +.75/e;
/* The next 3 contrast statements show one set of orthogonal contrasts*/
contrast "trt1 vs trt2-4" trt 3 -1 -1 -1;
contrast "trt2 vs trt3-4" trt 0 2 -1 -1 ;
contrast "trt3 vs trt4" trt 0 0 1 -1 ;
run;

```

The GLM Procedure

Class Level Information

Class	Levels	Values
trt	4	1 2 3 4
blk	4	1 2 3 4

Number of Observations Read 12

Number of Observations Used 12

The SAS System 12:49 Friday, August 15,
2008 2

The GLM Procedure

Dependent Variable: Y

Source	DF	Squares F	Sum of Mean Square	F Value	Pr >
Model	6	77.75000000 0.0024	12.95833333	19.94	
Error	5	3.25000000	0.65000000		
Corrected Total	11	81.00000000			

R-Square	Coeff Var	Root MSE	Y Mean
0.959877	1.112036	0.806226	72.50000

Source	DF	Type I SS F	Mean Square	F Value	Pr >
blk	3	55.00000000 0.0015	18.33333333	28.21	
trt	3	22.75000000 0.0107	7.58333333	11.67	

Source	DF	Type III SS F	Mean Square	F Value	Pr >
blk	3	66.08333333 0.0010	22.02777778	33.89	
trt	3	22.75000000	7.58333333	11.67	

The GLM Procedure
Least Squares Means

Coefficients for trt Least Square Means

Effect		trt Level			
		1	2	3	4
Intercept		1	1	1	1
blk	1	0.25	0.25	0.25	0.25
blk	2	0.25	0.25	0.25	0.25
blk	3	0.25	0.25	0.25	0.25
blk	4	0.25	0.25	0.25	0.25
trt	1	1	0	0	0
trt	2	0	1	0	0
trt	3	0	0	1	0
trt	4	0	0	0	1

trt	Y LSMEAN	Standard Error	Pr > t	LSMEAN Number
1	71.3750000	0.4868051	<.0001	1
2	71.6250000	0.4868051	<.0001	2
3	72.0000000	0.4868051	<.0001	3
4	75.0000000	0.4868051	<.0001	4

Least Squares Means for effect trt
Pr > |t| for H0: LSMean(i)=LSMean(j)

Dependent Variable: Y

i/j	1	2	3	4
1		0.7349	0.4117	0.0035
2	0.7349		0.6142	0.0047
3	0.4117	0.6142		0.0077
4	0.0035	0.0047	0.0077	

NOTE: To ensure overall protection level, only probabilities associated with pre-planned comparisons should be used.

The Mixed Procedure

Model Information

Data Set WORK.DATA1

Dependent Variable	Y
Covariance Structure	Variance Components
Estimation Method	REML
Residual Variance Method	Profile
Fixed Effects SE Method	Model-Based
Degrees of Freedom Method	Containment

Class Level Information

Class	Levels	Values
trt	4	1 2 3 4
blk	4	1 2 3 4

Dimensions

Covariance Parameters	2
Columns in X	5
Columns in Z	4
Subjects	1
Max Obs Per Subject	12

Number of Observations

Number of Observations Read	12
Number of Observations Used	12
Number of Observations Not Used	0

Iteration History

Iteration	Evaluations	-2 Res Log Like	Criterion
	0	1	44.37333968
1	1	34.22046396	0.00000000

Convergence criteria met.

The SAS System	12:49 Friday, August 15,
2008 5	

The Mixed Procedure

Covariance Parameter Estimates

Cov Parm	Estimate
blk	8.0167
Residual	0.6500

Fit Statistics

-2 Res Log Likelihood	34.2
AIC (smaller is better)	38.2
AICC (smaller is better)	40.6
BIC (smaller is better)	37.0

Type 3 Tests of Fixed Effects

Effect	DF	Num	Den	Pr > F
		DF	F Value	
trt	3	5	11.41	0.0113

Coefficients for trt effect 1

Effect	trt	Row1
Intercept		
trt	1	0.75
trt	2	-0.25
trt	3	-0.25
trt	4	-0.25

Coefficients for trt effect 2

Effect	trt	Row1
Intercept		
trt	1	-0.25
trt	2	0.75
trt	3	-0.25
trt	4	-0.25

The SAS System 12:49 Friday, August 15,
2008 6

The Mixed Procedure

Coefficients for trt effect 3

Effect	trt	Row1
Intercept		
trt	1	-0.25
trt	2	-0.25
trt	3	0.75
trt	4	-0.25

Coefficients for
trt effect 4

Effect	trt	Row1
Intercept		
trt	1	-0.25
trt	2	-0.25
trt	3	-0.25
trt	4	0.75

Estimates

Label	Estimate	Error	DF	Standard	
				t Value	Pr > t
trt effect 1	-1.0869	0.4269	5	-2.55	0.0515
trt effect 2	-0.8836	0.4269	5	-2.07	0.0932
trt effect 3	-0.5000	0.4269	5	-1.17	0.2942
trt effect 4	2.4705	0.4269	5	5.79	0.0022

Contrasts

Label	DF	Num		Den	Pr > F
		DF	F Value		
trt1 vs trt2-4	1	5	6.48	0.0515	
trt2 vs trt3-4	1	5	9.58	0.0270	
trt3 vs trt4	1	5	18.16	0.0080	

The SAS System 12:49 Friday, August 15,
2008 7

The Mixed Procedure

Coefficients for trt Least Squares Means

Effect	trt	Row1	Row2	Row3	Row4
Intercept					
		1	1	1	1
	trt	1		1	
	trt	2		1	
	trt	3			1
trt	4				1

Least Squares Means

Effect	trt	Estimate	Error	Standard		
				DF	t Value	Pr > t
trt	1	71.4131	1.4968	5	47.71	<.0001
trt	2	71.6164	1.4968	5	47.84	<.0001

trt	3	72.0000	1.4968	5	48.10	<.0001
trt	4	74.9705	1.4968	5	50.09	<.0001

Differences of Least Squares Means

Effect	trt	_trt	Estimate	Error	Standard DF	t Value	Pr > t
trt	1	2	-0.2033	0.6971	5	-0.29	0.7823
trt	1	3	-0.5869	0.6971	5	-0.84	0.4382
trt	1	4	-3.5574	0.6971	5	-5.10	0.0038
trt	2	3	-0.3836	0.6971	5	-0.55	0.6058
trt	2	4	-3.3541	0.6971	5	-4.81	0.0048
trt	3	4	-2.9705	0.6971	5	-4.26	0.0080

Note that the least squares means for treatments when using PROC Mixed, correspond to the combined intra- and inter-block estimates of the treatment effects.

Random Effect Factor

So far we have discussed experimental designs with fixed factors, that is, the levels of the factors are fixed and constrained to some specific values. However, this is often not the case. In some cases, the levels of the factors are selected at random from a larger population. In this case, the inference made on the significance of the factor can be extended to the whole population but the factor effects are treated as contributions to variance.

Minitab's General Linear Command handles random factors appropriately as long as you are careful to select which factors are fixed and which are random.